

Exercise 1:

Fill out the following DH table using the QArm values

Link	θ_i	d_i	a_i	α_i
1				
2				
3				
4				

Exercise 2: Using your DH table, calculate the full transformation matrix from frame $\{0\}$ to $\{4\}$ (0T_4); feel free to use a calculator or computer. Keep this recorded, as we will use it later.

Tip: Find the intermediate transformation matrices, then multiply them together. We recommend implementing a function such as: `find_matrix_DH(theta, d, a, alpha)`.

Exercise 3: Test your function using the following inputs and expected outputs:

Input	Location [x, y, z]	Rotation Matrix
[0, 0, 0, 0]	[0.45, 0.0, 0.49]	$\begin{pmatrix} 0.0 & 0.0 & 1.0 \\ 0.0 & -1.0 & 0.0 \\ 1.0 & 0.0 & 0.0 \end{pmatrix}$
[30, 55, -60, 0]	[-0.08, 0.5, 0.86]	$\begin{pmatrix} -0.08 & 0.5 & 0.86 \\ -0.04 & -0.87 & 0.5 \\ 1.0 & 0.0 & 0.09 \end{pmatrix}$

Debug any issues that arise, record your results for the following inputs:

Input	Location [x, y, z]	Rotation Matrix
[0, -11, -75, 0]		
[1, 2, 3, 4]		
[1, 2, 3, 4]		
[1, 2, 3, 4]		

Exercise 4:

Fill out joint positions for each point of interest

		Position			
		home	1	2	3
Joint	base				
	shoulder				
	elbow				
	wrist				

Exercise 5:

Calculate and record the task space positional error (magnitude) in several poses when we purposefully introduce error using your forward kinematics implementation.

Pose_1 = [0, 0, 0, 0] (home)

Pose_2 = [0, 55, -60, 0] (extended)

Pose_3 = [0, -80, 70, 0] (retracted)

Error Type	Pose 1 Positional Error	Pose 2 Positional Error	Pose 3 Positional Error
Shoulder joint + 10°			
Elbow joint + 10°			
L2 length + 5cm			
L4 length + 5cm			

Question: Identify which joint and link errors cause the largest overall positional discrepancy. Why does an error at one specific joint or link scale differently than another? Based on this behavior, what is one additional distinct pose you would test to explore the absolute worst-case positional error?

Exercise 6: Test your function using the following inputs and expected outputs:

Input	J	J_inv
$[0, 0, 0, 0]$	$\begin{pmatrix} -0.0 & 0.08 & -0.22 & 0.0 \\ 0.53 & 0.0 & -0.0 & 0.0 \\ 0.0 & -0.53 & -0.34 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{pmatrix}$	$\begin{pmatrix} 0.0 & 1.9 & 0.0 & 0.0 \\ 2.38 & -0.0 & -1.53 & -0.0 \\ -3.73 & -0.0 & -0.58 & -0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{pmatrix}$
$[90, 55, -60, 0]$	$\begin{pmatrix} 0.06 & 0.2 & 0.14 & 0.0 \\ 0.03 & -0.39 & -0.27 & 0.0 \\ 0.0 & 0.06 & -0.26 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{pmatrix}$	$\begin{pmatrix} 14.45 & 7.24 & 0.0 & 0.0 \\ 0.87 & -1.74 & 2.24 & -0.0 \\ 0.21 & -0.41 & -3.28 & -0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{pmatrix}$
$[-30, -80, 70, 90]$	$\begin{pmatrix} -0.06 & -0.09 & -0.05 & 0.0 \\ 0.01 & -0.55 & -0.33 & 0.0 \\ 0.0 & -0.06 & 0.21 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{pmatrix}$	$\begin{pmatrix} -16.48 & 2.57 & -0.0 & -0.0 \\ -0.24 & -1.51 & -2.4 & -0.0 \\ -0.07 & -0.42 & 3.99 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{pmatrix}$

Debug issues that arise; when complete, show these to the TA as evidence of completion.

Exercise 7: A trajectory with defined knot points is provided in [Insert File Name]. Implement trajectory generation using your inverse Jacobian function and `scipy.interpolate.CubicSpline`. The knot points are formatted as 2D NumPy arrays, where each sub-array contains three values representing the (x, y, z) coordinates expressed in the base $\{0\}$ frame.

Exercise 8: While cubic spline trajectories are convenient and straightforward to implement, what are their main mechanical and kinematic drawbacks?